ANÁLISE DE PROJETO E SISTEMAS - PROJETO FINAL

Da Concepção ao Dia 1 do Dev

Tudo que acontece antes da primeira linha de código.
Engenharia de software aplicada do zero ao handoff completo.

Entrega: 1 semana

Grupos: Trios

Apresentacão

Toda decisao justificada

O que vocês vão entregar

NÃO e implementar. E arquitetar, projetar e documentar como uma equipe técnica real faz antes do código.

- V **Nao e implementar:** E projetar, arquitetar e documentar o sistema completo antes de qualquer código.
- V **Decisoes tecnicas justificadas:** banco, stack, arquitetura, infra, tudo comparado com a alternativa descartada.
- V **O projeto é uma coisa só:** cada bloco depende do anterior. Inconsistencia entre partes perde nota.
- V **Entregaveis final:** um projeto tão completo que um dev recebe e começa no dia seguinte, sem respostas.

Tudo que foi aprendido na disciplina, engenharia de software, UML, banco, arquitetura vão aparecer aqui.

Um bloco depende do anterior

NÃo são 9 documentos independentes. E um projeto único e coerente - cada decisão alimenta a próxima.

Problema Bloco 1 -> **Requisitos** Bloco 2 -> **Arquitetura** Bloco 3 -> **UML** Bloco 4 -> **Banco** Bloco 5 -> **Infra** Bloco 6 -> **Custo** Bloco 7 -> **Dev pronto** Bl.8-9

Se vocês definiram...	Então obrigatoriamente...
RNF: 99,9% de uptime	A infra precisa ter load balancer, redundancia e failover
Microservicos na arquitetura	O banco nao pode ser unico monolitico, cada servico tem seu BD
PostgreSQL como banco	Modelo relacional com FKs, constraints e normalizacao correta
LGPD nos RNFs	Dicionario de dados marca campos com tratamento definido
3 sprints no planejamento	Estimativa de esforco precisa caber com a equipe definida

ATENCAO: Inconsistencia entre blocos = perda de nota, mesmo que cada parte individualmente pareca correta.

Equipes e Papeis

Trios. Cada integrante assume um papel, mas todos defendem o projeto inteiro na arguição.

PO

Analista / PO

Visão do produto, personas, requisitos funcionais e não funcionais, historias de usuario, criterios de aceite, priorizacao MoSCoW

ARQ

Arquiteto de Software

ADR, Modelo C4 (4 niveis), stack tecnologica, padroes de design, todos os diagramas UML, coerencia da arquitetura

DBA

Engenheiro / DBA

Banco de dados (conceitual ao DDL Data Definition Language), infraestrutura cloud, pipeline CI/CD, seguranca OWASP, estimativas e custo real

Modelo C4 é uma abordagem de abstração hierárquica para documentar e diagramar arquiteturas de software de forma clara e escalável.

ADR - Architecture Decision Record (Registro de Decisão de Arquitetura)

REGRA DA ARGUICAO: eu vou escolher qualquer integrante e pergunta sobre qualquer parte. Não existe 'essa parte foi do fulano'.

TEMA

Escolha um domínio

Minimo: 4 atores e 12 funcionalidades. Sistema generico (cadastro de clientes) nao sera aceito.

Fintech
Carteira digital, pagamentos, controle financeiro, credito

Saude
Telemedicina, prontuario eletronico, agendamento hospitalar

Logistica
Rastreamento de entregas, estoque, roteirizador

EdTech
Plataforma de cursos, gestao escolar, provas, mentoria

Marketplace
Compra/venda, leilao, aluguel de servicos

GovTech
Ouvidoria pública, mobilidade inteligente, transparencia

BLOCO 1 - OBRIGATORIO

Concepção e Escopo

O que vamos construir e por que? A base que define tudo que vem depois.

1. **Documento de Visao:** nome, slogan, problema real, proposta de valor, o que está fora do escopo
2. **Personas (min. 3):** nome, idade, contexto profissional e motivação real baseadas no domínio
3. **Mapa de Stakeholders:** quem sao, qual interesse tem e qual influência exercem sobre o projeto
4. **Estudo de Viabilidade:** técnica + operacional + custo de infra mensal com valores reais (AWS/GCP/Azure)
5. **Riscos iniciais (min. 5):** cada risco com probabilidade, impacto e plano de mitigacao

1

Escopo franco = arquitetura vaga. Se o problema nao for claro aqui, nada que vem depois será coerente.

BLOCO 2 - OBRIGATORIO

Requisitos

O que o sistema deve fazer e como deve se comportar? Toda metrica precisa de numero.

RF **Requisitos Funcionais (min. 20):** numerados RF001..., com prioridade MoSCoW (Must/Should/Could/Wont) + justificativa

RNF **RNFs com metrica obrigatoria:** desempenho (<300ms p95), escalabilidade (10k users), seguranca, LGPD

US **Historias de usuario (min. 15):** Como [persona], quero [acao], para [objetivo] + criterios Given/When/Then + story points

2

ERRADO: 'O sistema deve ser rapido / seguro'. Todo RNF precisa de numero mensuravel.

BLOCO 3 - OBRIGATORIO

Arquitetura de Software

Como o sistema vai ser estruturado? Cada decisão precisa de ADR sem exceção.

ADR **Architecture Decision Record:** contexto, decisao, alternativas descartadas, consequencias. Um ADR por decisão relevante.

C4 **Modelo C4 -> 4 niveis obrigatorios:** Contexto, Containers (tecnologias, protocolos), Componentes, Codigo (1 componente critico)

ST **Stack tecnologica:** tabela: camada x escolha x alternativa descartada x motivo técnico. Todas as camadas.

DP **Padroes de design:** quais patterns aplicados (Repository, Factory, CQRS, Saga...), onde e por que

3

ERRADO: 'Escolhemos React porque e popular / conhecemos'. O motivo precisa derivar dos requisitos.

BLOCO 4 - OBRIGATORIO

Diagramas UML

6 diagramas obrigatorios - todos apresentados ao vivo com explicacao. Titulo + autores + data.

UC **Casos de Uso:** todos os atores, casos de uso, relacoes include / extend / generalização

CL **Classes (min. 10):** atributos tipados (nome: String), metodos, visibilidades (+/-/#), relações corretas

SQ **Sequencia x3:** autenticacao, fluxo principal (happy path), cenario de erro critico

CP **Componentes:** modulos, interfaces expostas, dependencias internas e sistemas externos

DP **Deployment:** infra cloud, ambientes dev/staging/prod, protocolos e portas

AT **Atividades x2:** processo de negócio principal + processo administrativo relevante

4

BLOQUEADOR: Classes sem tipos dos atributos zera o bloco. Diagrama na tela sem explicacao = nao apresentado.

BLOCO 5 - OBRIGATORIO

Banco de Dados

Do modelo conceitual ao executável. A escolha do banco precisa vir da arquitetura.

BD **Decisao do banco:** tabela: banco escolhido x alternativa descartada x motivo tecnico baseado nos requisitos

ER **Modelo Conceitual (ER):** todas as entidades do domínio com cardinalidades corretas

LG **Modelo Logico:** tabelas com atributos, tipos, PKs, FKs e indices justificados

FI **Modelo Fisico :** CREATE TABLE das tabelas principais com constraints - precisa ser executável

DC **Dicionario de Dados:** campo, tipo, restrições e descrição de negócio por entidade

ST **Estrategia:** backup (frequencia + retencao), LGPD (campos marcados), migrations sem downtime

BLOCO 6 - OPCIONAL

Infraestrutura e DevOps

Onde o sistema roda e como chega lá. Custos precisam ser reais - use calculadoras cloud.

IN **Diagrama de infra:** VPC, subnets, load balancer, instancias, banco, storage. Custo mensal real por ambiente.

CI **Pipeline CI/CD completo:** lint > testes > build > seguranca > staging > smoke test > prod. Criterio de bloqueio definido.

DK **Containerizacao:** Dockerfile por servicio (multi-stage build) + Docker Compose para desenvolvimento local

SE **Seguranca:** autenticacao (JWT/OAuth2/OIDC), RBAC/ABAC, secrets management, OWASP Top 10 enderecado

OB **Observabilidade:** logging estruturado, metricas de negocio e tecnicas, alertas configurados

6

NÃO BLOQUEADOR: Custo inventado sem calculo real = bloco zerado. Use calculator.aws ou cloud.google.com/calculator.

BLOCO 7 - OBRIGATORIO

Planejamento e Estimativas

Quanto vai custar, quanto tempo e quais os riscos? Numeros reais de mercado.

WB **WBS (Work Breakdown Structure):** hierarquia completa de entregas, minimo 3 niveis de decomposicao

ES **Estimativa de esforco:** funcionalidades com story points, velocidade da equipe, cronograma em sprints de 2 semanas

EQ **Composicao da equipe de dev:** perfis necessarios, quantos, custo mensal real (Glassdoor), custo total do projeto

RI **Matriz de Riscos (min. 8):** tecnicos, negocio, equipe, seguranca, regulatorios — probabilidade x impacto x mitigacao

7

BLOQUEADOR: Custo inventado ou equipe sem pesquisa salarial real = bloco zerado.

BLOCOS 8 E 9 - OBRIGATORIO

Documentacao para o Dev + Qualidade

O dev chega no dia 1. O que ele precisa para começar sem fazer uma pergunta?

BLOCO 8 -> Documentacao Dev

-> README completo (clone ao localhost)
-> Variaveis de ambiente (.env.example)
-> Contrato de API - OpenAPI 3.0
-> Guia de padroes de codigo
-> Git workflow definido
-> Sprint 1 detalhado com criterios de aceite

BLOCO 9 -> Qualidade e Processo

-> Modelo de Dominio / glossario
-> Estrategia de testes (unitario, integracao, E2E)
-> Metas de cobertura de codigo
-> Definition of Ready (DoR)
-> Definition of Done (DoD)
-> Metodologia de dev (Scrum/Kanban justificado)

CRITERIO DE SUCESSO: um desenvolvedor que receba esse projeto começa a trabalhar no dia seguinte sem fazer nenhuma pergunta.

REGRA FUNDAMENTAL

Toda e qualquer decisão precisa de justificativa

Comparando sempre com a alternativa descartada — em todos os 9 blocos, sem excecao.

Bloco	Decisao que precisa de justificativa técnica
Requisitos	Por que esse RNF e Must? Por que essa métrica e não outra?
Arquitetura	Por que microservicos e nao monolito? Por que REST e nao gRPC?
UML	Por que composicao aqui e nao agregação?
Banco	Por que PostgreSQL e nao MongoDB? Por que normalizacao ate 3FN?
Infra	Por que AWS? Por que Kubernetes e nao ECS ou Lambda?
Planejamento	Por que esse numero de sprints? Por que esse perfil de equipe?

INVALIDO: 'Escolhemos porque conhecemos' | 'E o mais popular' | 'Todo mundo usa'
Nenhuma dessas e justificativa técnica válida.

ATENCAO

O que zera o bloco

- X **Arquitetura sem ADR ->** 'escolhemos porque gostamos / conhecemos'
- X **Diagrama de classes sem tipos ->** atributo sem tipo (so 'nome' sem 'nome: String') e sem visibilidade (+/-/#)
- X **Banco sem comparação ->** escolha sem tabela de alternativa descartada
- X **RNFs sem metricas ->** 'o sistema deve ser rapido / seguro / escalavel' sem numero
- X **Custo inventado ->** número de infra sem cálculo real nas calculadoras de cloud
- X **Inconsistencia entre blocos ->** arquitetura define microservicos mas banco e unico monolitico
- X **Diagrama sem explicação ->** colocar na tela e passar rápido = não foi apresentado
- X **Arguição sem resposta ->** integrante que não consegue explicar a própria parte

ENTREGA ANTES DA APRESENTACAO

Documento de Apoio

Até +1,5 PONTO

Entregue antes do início das apresentações. Vale até 1.5 ponto na nota final unidade 2.

- V **Conteudo:** todos os 9 blocos documentados por escrito, com todos os diagramas incluídos e legíveis
- V **Formato:** PDF bem estruturado com sumário, organizado na ordem dos 9 blocos
- V **Cada diagrama:** titulo, nomes dos integrantes e data obrigatórios
- V **Prazo:** entregue antes do primeiro grupo apresentar

O AID (Documento de apoio) não substitui a apresentação. Se um diagrama está no AID mas não é apresentado e explicado ao vivo, não conta.

AID

APRESENTACAO

Estrutura da Apresentação

10 minutos de apresentacao + 1 minuto de arguicao. Tudo que foi feito aparece na tela.

Conteudo	Quem apresenta
Produto, requisitos, personas, MoSCoW, RNFs com metricas, historias de usuario	Analista / PO
ADR, C4 (4 niveis), stack justificada, Casos de Uso, Classes, Sequencias (x3), Componentes	Arquiteto
Decisão do banco, ER > Logico > DDL, infra cloud, pipeline CI/CD, seguranca OWASP	DBA / Infra
Estimativas, custo real, riscos, README aberto — 'o dev pode comecar amanha?'	Todos
Arguição - Eu pergunto para qualquer integrante sobre qualquer parte	Todos

ATENCAO: Diagrama na tela sem explicacao = nao apresentado. Tudo que aparecer precisa ser falado.

ARGUICAO

Perguntas que poderão ser feitas

Eu escolho qualquer pessoa. Não existe 'essa parte foi do colega'.

? **Por que voces nao usaram MongoDB nesse sistema?**
? **Se a demanda triplicar, o que muda na arquitetura?**
? **O que acontece se o banco de dados cair? A infra de vocês cobre isso?**
? **Esse diagrama de classes tem herança aqui, por que não usou composição?**
? **Voces marcaram LGPD nos RNFs. No dicionario de dados, quais campos são PII?**
? **Se precisar adicionar pagamento por PIX depois, o que muda na arquitetura?**
? **O SLA de vocês e 99,9%. A infra desenhada garante isso como?**

Ensaiem: cada integrante explica qualquer bloco para os outros antes da apresentação.

AVALIACAO

Distribuição de Pontos

Bloco	O que sera avaliado
1 - Concepção e Escopo	Problema claro, personas reais, viabilidade com custo real
2 - Requisitos	Completude, RNFs mensuraveis, criterios de aceite
3 - Arquitetura	fundamentação, niveis, stack justificada
4 - UML	Correcao tecnica, cobertura dos fluxos principais e de erro
5 - Banco de Dados	Conceitual > Logico > Fisico corretos, funcional, LGPD
6 - Infraestrutura	Pipeline CI/CD real, seguranca OWASP, custo com valores reais
7 - Planejamento	Estimativas realistas, riscos reais, custo de equipe de mercado
8+9 - Doc + Qualidade	README funcional, DoR/DoD, estrategia de testes
Entrega previa	Documento entregue antes das apresentacoes

FERRAMENTAS

O que usar

A escolha da ferramenta e livre. O que importa e a qualidade e correção do conteúdo.

UML e C4

-> draw.io - app.diagrams.net

-> Lucidchart

-> PlantUML (codigo)

-> Structurizr (C4)

Banco de Dados

-> dbdiagram.io

-> MySQL Workbench

-> pgAdmin

Documentacao

-> Notion

-> Google Docs

-> Swagger Editor (API)

Custos reais

-> calculator.aws

-> cloud.google.com/calculator

-> glassdoor.com.br

CRONOGRAMA

1 semana — o que focar em cada dia

Dia 1	**Concepcao:** dominio, sistema, papeis, Visão do Produto, Stakeholders, Personas
Dia 2	**Requisitos:** RFs com MoSCoW, RNFs com metricas reais, historias de usuario com Given/When/Then
Dia 3	**Arquitetura:** ADR(Architecture Decision Record), stack justificada, Casos de Uso, Diagrama de Classes
Dia 4	**UML + Banco:** Sequencias (x3), Componentes, Deployment, Atividades. ER > Modelo Logico > DDL (Data Definition Language)
Dia 5	**Infra + Planejamento:** CI/CD, Seguranca OWASP, Custo real, Riscos, WBS, DoR/DoD, README
Dia 6	**AID + Ensaio:** montar PDF, verificar coerencia entre blocos, ensaiar todos explica tudo
Dia 7	**APRESENTACAO + Entrega**

CRITERIO DE SUCESSO

Um desenvolvedor real pode começar amanhã.

Se o dev chega no dia 1 sem precisar perguntar nada, arquitetura clara, banco definido, infra mapeada, backlog pronto, o projeto está completo.

9 blocos coerentes

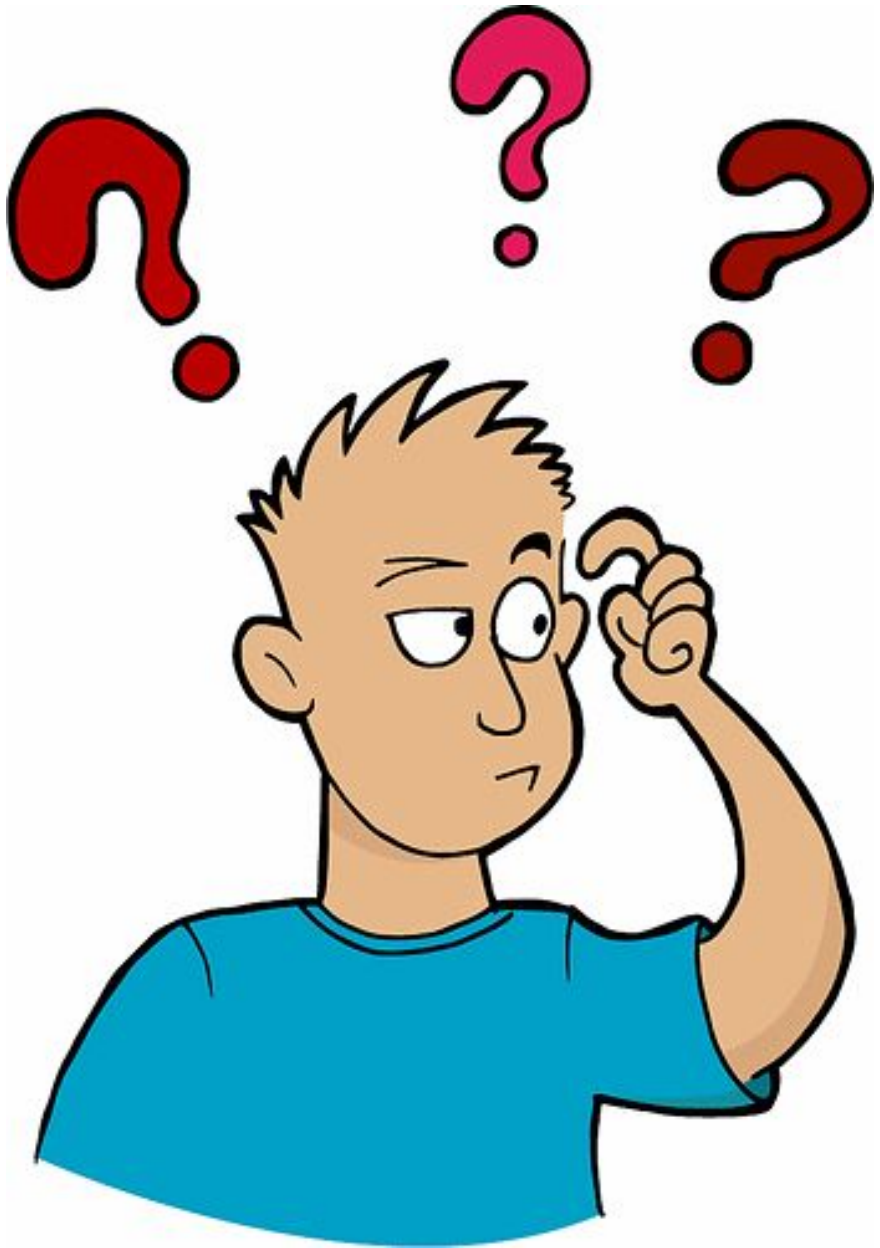
Toda decisao justificada

Cada bloco depende do anterior

30 min + arguicao

AID vale até 1.5 pontos

UNIPÊ
Centro Universitário
de João Pessoa



OBRIGADO!



B.Sc. SI - Israel Cunha
Pós-graduado Testes Ágeis
Cesar School - REC